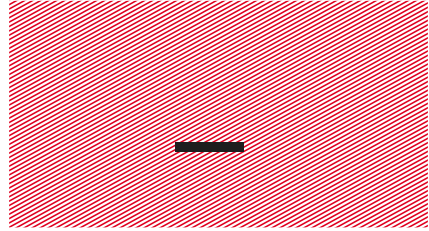


— IoT Devices

Les 7 – RTOS deel 1





RTOS

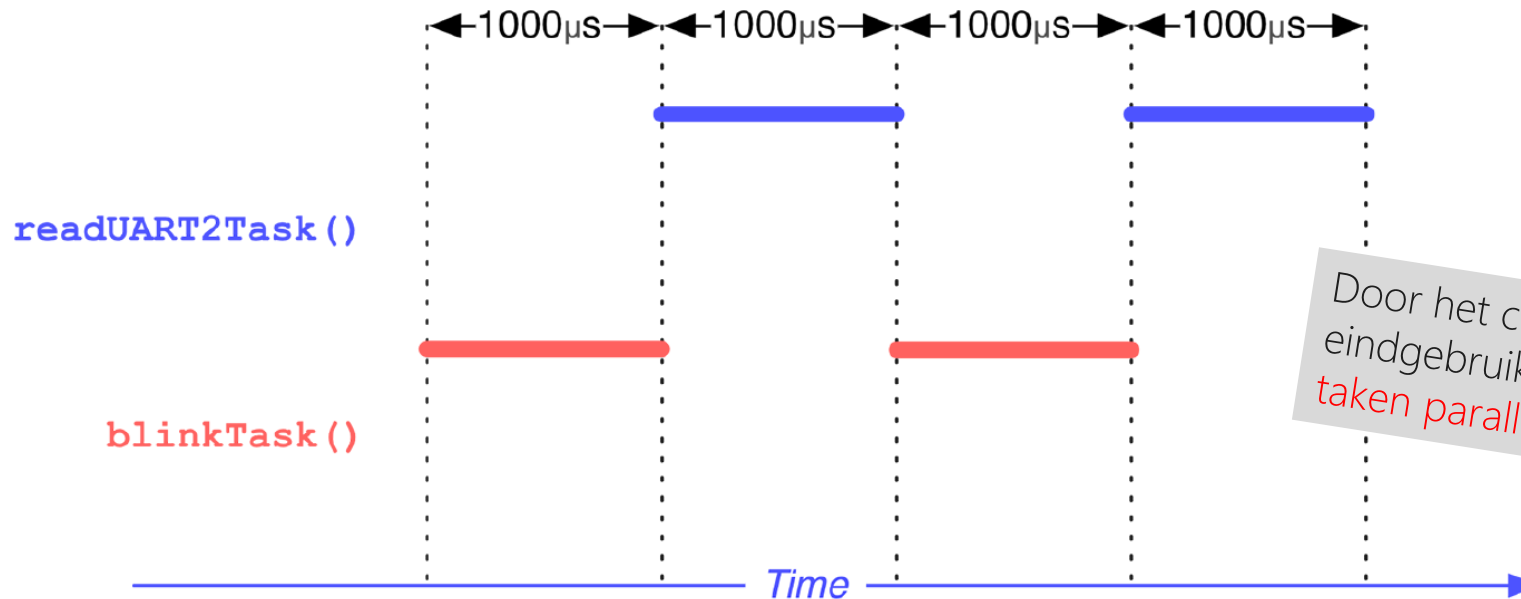
algemene info



- (Free)RTOS is een “preemptive multitasking operating system”.
- **Preemptive** = voorrang verlenen aan...
- In een microcontroller met RTOS, krijgen verschillende taken (tasks) een wel bepaald **tijdslot** (time slice) om in te werken. Daarna verlenen die taken **voorrang** aan een andere taak... Enzovoort...
- Een RTOS bevat een ‘**scheduler**’ die het uitvoeren van de taken plant.

RTOS algemene info

- Het activeren of deactiveren van een taak gaat gepaard met een '**context switch**'. Er wordt een '**snapshot**' genomen van de huidige taaktoestand. Dat snapshot wordt later gebruikt om terug te keren naar diezelfde taak...

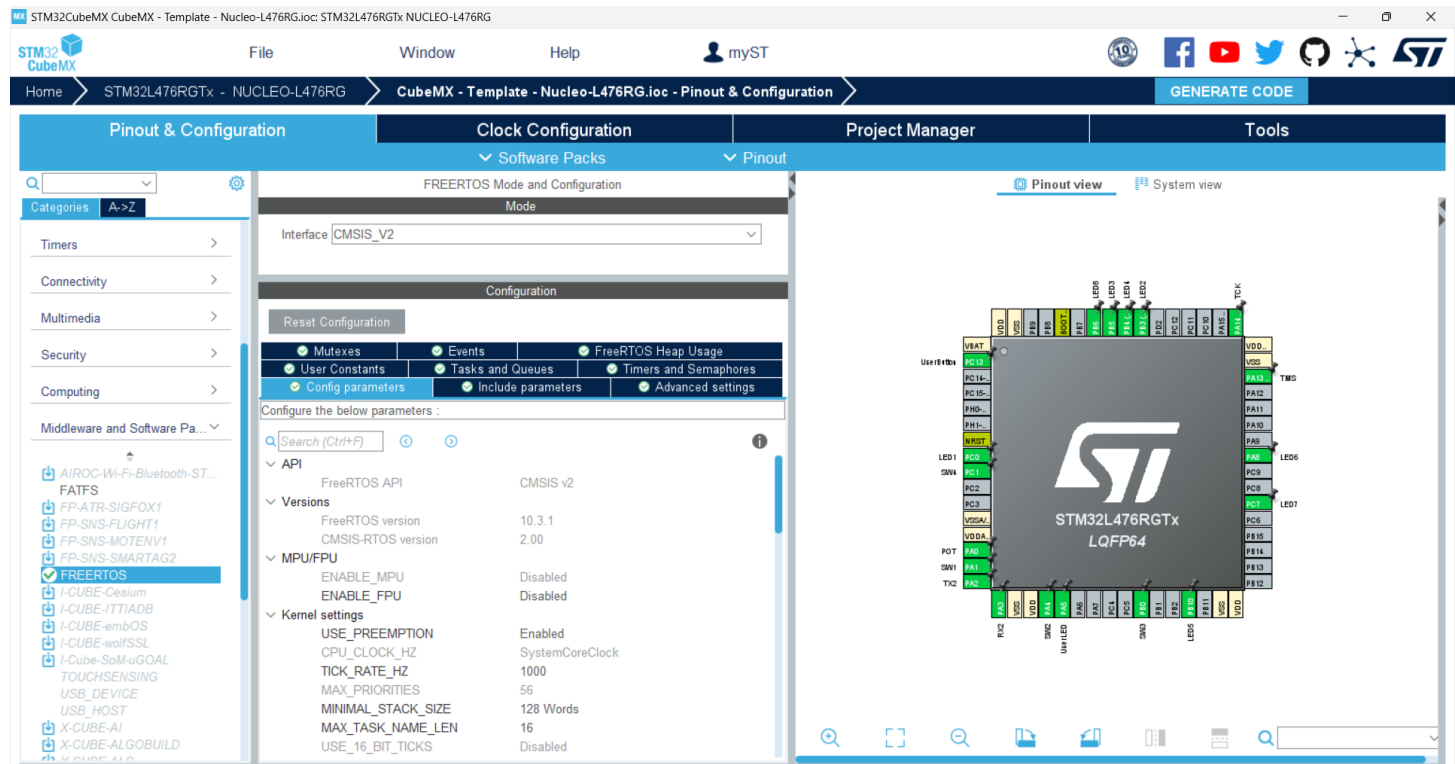


Door het continu switchen, krijgt de eindgebruiker de indruk dat alle **taken parallel uitgevoerd** worden...!

- Het 'snapshot' uit de vorige slide, wordt eigenlijk beter '**Thread Control Block**' (TCB) genoemd.
- In zo'n TCB worden de variabelen van die thread, de program counter, enkele CPU-registers en andere zaken **bewaard in RAM**.
- De context switching gebeurt normaal gezien **op het ritme van de SysTick** timer (1ms).
- Als we **HAL-functies** willen gebruiken, zullen we daardoor de SysTick timer niet meer beschikbaar hebben. Kies dan bijvoorbeeld **Timer 6** daarvoor.

RTOS algemene info

- Wij zullen **CMSIS RTOS V2** gebruiken. Dat is FreeRTOS 'gewrapped' in CMSIS code. Dat zou het 'porten' nog moeten vereenvoudigen.
- Deze CMSIS RTOS V2 wordt goed ondersteund door **STM32CubeMX**!



RTOS algemene info

Officiële info over CMSIS RTOS V2?

https://arm-software.github.io/CMSIS_6/latest/RTOS2/group_CMSIS_RTOS.html

The screenshot shows the CMSIS-RTOS2 API documentation page. The header includes the CMSIS logo, the title 'CMSIS-RTOS2 Version 2.3.0', and the subtitle 'Real-Time Operating System API'. A navigation bar contains tabs for Overview, Core, Driver, RTOS2 (selected), DSP, NN, View, Compiler, Toolbox, Stream, DAP, and Zone. A left sidebar lists the API Reference structure, with 'CMSIS-RTOS2 API' expanded to show sub-sections like Kernel Information and Control, Thread Management, Thread Flags, Event Flags, Generic Wait Functions, Timer Management, Mutex Management, Semaphores, Memory Pool, Message Queue, Generic Definitions, and OS Tick API. The main content area, titled 'CMSIS-RTOS2 API', describes the C interface of the CMSIS-RTOS2 API defined in cms_os2.h. It lists the following sections: Kernel Information and Control (Provides version/system information and starts/controls the RTOS Kernel), Thread Management (Define, create, and control thread functions), Thread Flags (Synchronize threads using flags), Event Flags (Synchronize threads using event flags), Generic Wait Functions (Wait for a certain period of time), and Timer Management.

RTOS algemene info

- Een **Task** (of thread) is in feite niets anders dan een **C-functie**.

```
394 void StartDefaultTask(void *argument)
395 {
396     /* USER CODE BEGIN 5 */
397     /* Infinite loop */
398     for(;;)
399     {
400         // Binaire teller maken.
401         ByteToLeds(count++);
402
403         // Even wachten.
404         osDelay(100);
405     }
406     /* USER CODE END 5 */
407 }
```

Naast `while(1)` is `for(;;)` een veel gebruikte manier in C-taal om een oneindige lus te maken.

Let op: gebruik geen `HAL_Delay()` meer!

RTOS algemene info

Het **aanmaken** van zo'n **thread**, gebeurt met volgende functie:

```
osThreadId_t osThreadNew ( osThreadFunc_t    func,  
                           void *            argument,  
                           const osThreadAttr_t * attr  
                           )
```

Kan je opzoeken wat er in
een `osThreadAttr_t` zit?

Parameters

- [in] **func** thread function.
- [in] **argument** pointer that is passed to the thread function as start argument.
- [in] **attr** thread attributes; NULL: default values.

Returns

thread ID for reference by other functions or NULL in case of error.

The function **osThreadNew** starts a thread function by adding it to the list of active threads and sets it to state **READY**. Arguments for the thread function are passed using the parameter pointer **argument*. When the priority of the created thread function is higher than the current **RUNNING** thread, the created thread function starts instantly and becomes the new **RUNNING** thread. Thread attributes are defined with the parameter pointer *attr*. Attributes include settings for thread priority, stack size, or memory allocation.

The function can be safely called before the RTOS is started (call to **osKernelStart**), but not before it is initialized (call to **osKernelInitialize**).

The function **osThreadNew** returns the pointer to the thread object identifier or **NULL** in case of an error.

Bron: https://arm-software.github.io/CMSIS_6/latest/RTOS2/group_CMSIS_RTOS_ThreadMgmt.html#ga48d68b8666d99d28fa646ee1d2182b8f

In ons concreet voorbeeld:

```
52 /* Definitions for defaultTask */
53 osThreadId_t defaultTaskHandle;
54 const osThreadAttr_t defaultTask_attributes = {
55     .name = "defaultTask",
56     .stack_size = 128 * 4,
57     .priority = (osPriority_t) osPriorityNormal,
58 };
```

```
133 /* Create the thread(s) */
134 /* creation of defaultTask */
135 defaultTaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);
```

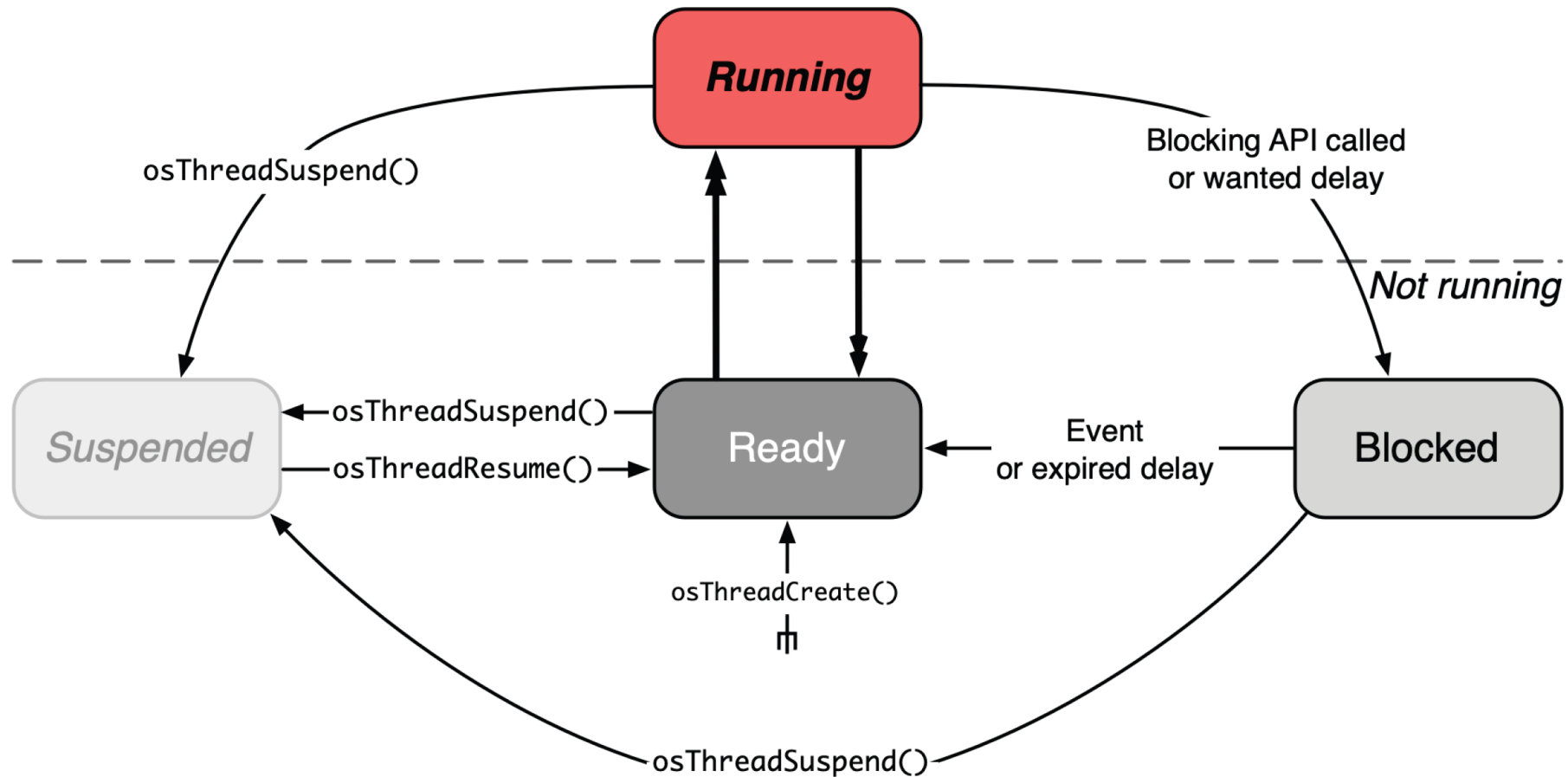
OPM: * 4 om uitlijning op 32 bit en compatibiliteit met FreeRTOS te bekomen.

De grootte bepalen van de *stack_size* is dikwijls **niet eenvoudig**. Veelal wordt die experimenteel bepaald...

Ze hangt af van lokale variabelen, parameters, optimalisatieniveau, compiler, diepte van de call stack, ...

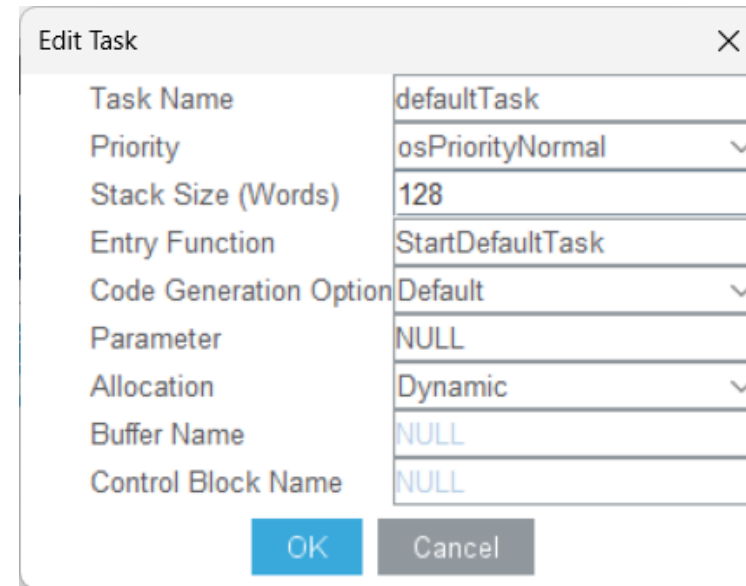
- Een **thread** kan zich in **vier** situaties/**states** bevinden:
 1. Running
 2. Ready
 3. Blocked
 4. Suspended
- OPM: de suspended state wordt zelf aangevraagd door de thread en komt daardoor eigenlijk ook in de blocked state terecht.

Thread States



RTOS algemene info

- De **prioriteiten** per task zijn **instelbaar**. Ook daarbij kan STM32CubeMX helpen.



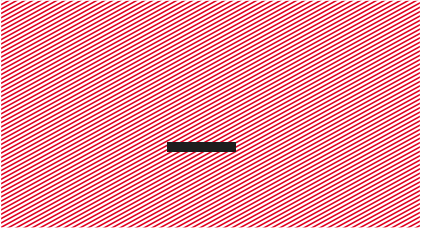
Field	Value
Task Name	defaultTask
Priority	osPriorityNormal
Stack Size (Words)	128
Entry Function	StartDefaultTask
Code Generation Option	Default
Parameter	NULL
Allocation	Dynamic
Buffer Name	NULL
Control Block Name	NULL

- STM32CubeMX zal standaard kiezen voor 'prioritized preemptive scheduling with time slicing'. Daardoor krijgen taken met **gelijke prioriteit**, elk **om de beurt een tijdslot**.



RTOS algemene info

- Wat als geen enkele thread tijd nodig heeft? Dan is er de 'idle thread'... Die zorgt dat de CPU ' bezig gehouden wordt ' tot een andere thread de aandacht opeist.
- De idle thread heeft de laagste prioriteit en wordt automatisch aangemaakt.



Compiler V6 gebruiken

Compiler V6 gebruiken

- Onze **µVision** gebruikt standaard 'Compiler **V6**'. Maar... vanuit **STM32CubeMX** wordt ervan uitgegaan dat je 'Compiler **V5**' zal gebruiken. Hierdoor staan sommige paden naar bestanden niet correct.
- Omdat we toch met **STM32CubeMX** en **Compiler V6** willen werken, moet je de 'work around' van op de volgende slides toepassen.
- **Tip:** als je start van een RTOS template project, zal je dat **niet telkens opnieuw** moeten doen...



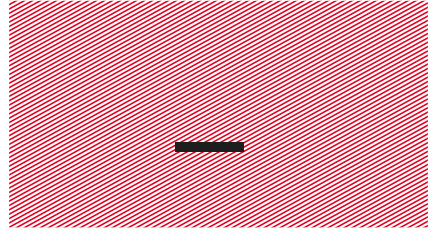
Compiler V6 gebruiken

Aanmaken van een nieuw **FreeRTOS project met Compiler V6**:

1. In **STM32CubeMX** gewoon het **project maken**, met normale verwijzing naar FreeRTOS (via middleware).
2. In STM32CubeMX de 'Tool chain IDE' **NIET** op MDK-ARM zetten, maar op '**makefile**'. Genereer het project.

OPM: Dit zorgt ervoor dat de juiste FreeRTOS bestanden in de 'FreeRTOS/Source/portable/**GCC**' map terecht komen.

3. Zet nu de 'Tool chain IDE' wel **terug op 'MDK-ARM'**. Genereer. Dit genereert een μ Vision project, **maar met foute verwijzing (RVDS)** naar de FreeRTOS bestanden.
4. Open het ***.uvprojx** bestand in **kladblok** en wijzig elke 'RVDS' in de paden naar 'GCC'.
5. Nu kan je het project (terug) in **μ Vision** openen.
6. **Compiler 6 kiezen** via 'Options – Target'.
7. Indien nodig, voeg het juiste include pad toe via Options - C++:
`../Middlewares/Third_Party/FreeRTOS/Source/portable/GCC/ARM_CM4F.`



Blinky blinky





Blinky blinky

Doel?

Een **basisvoorbeeld maken** met CMSIS RTOS V2.

Hoe?

Gebruik STM32CubeMX om een CMSIS RTOS V2 project aan te maken met twee threads.

Beide threads laten een **LED knipperen** op een andere frequentie...

De eindgebruiker krijgt de indruk dat beide threads simultaan/parallel lopen.

Blinky blinky

STM32CubeMX CubeMX - Template - Nucleo-L476RG.ioc: STM32L476RGTx NUCLEO-L476RG

File Window Help myST

Home > STM32L476RGTx - NUCLEO-L476RG > CubeMX - Template - Nucleo-L476RG.ioc - Pinout & Configuration

Pinout & Configuration Clock Configuration Project Manager

Software Packs Pinout

FREERTOS Mode and Configuration

Mode

Interface CMSIS_V2

Configuration

Reset Configuration

Tasks and Queues Timers and Semaphores Mutexes Events FreeRTOS Heap Usage

Config parameters Include parameters Advanced settings User Constants

Configure the below parameters :

Search (Ctrl+F)

API

FreeRTOS API

Versions

FreeRTOS version 10.3.1

CMSIS-RTOS version 2.00

CMSIS v2

—

Blinky blinky

✓ Tasks and Queues

✓ Timers and Semaphores

✓ Mutexes

✓ Events

✓ FreeRTOS Heap Usage

✓ Config parameters

✓ Include parameters

✓ Advanced settings

✓ User Constants

Tasks

Task Name	Priority	Stack Size ...	Entry Function	Code Generatio...	Parameter	Allocation	Buffer Name	Control Block ...
BlinkLed1Thread	osPriorityNormal	128	BlinkLed1	Default	NULL	Dynamic	NULL	NULL
BlinkLed2Thread	osPriorityNormal	128	BlinkLed2	Default	NULL	Dynamic	NULL	NULL

Blinky blinky

```
52  /* Definitions for BlinkLed1Thread */
53  osThreadId_t BlinkLed1ThreadHandle;
54  const osThreadAttr_t BlinkLed1Thread_attributes = {
55      .name = "BlinkLed1Thread",
56      .stack_size = 128 * 4,
57      .priority = (osPriority_t) osPriorityNormal,
58  };
59  /* Definitions for BlinkLed2Thread */
60  osThreadId_t BlinkLed2ThreadHandle;
61  const osThreadAttr_t BlinkLed2Thread_attributes = {
62      .name = "BlinkLed2Thread",
63      .stack_size = 128 * 4,
64      .priority = (osPriority_t) osPriorityNormal,
65  };
```



Blinky blinky

```
125  /* Init scheduler */
126  osKernelInitialize();

144  /* Create the thread(s) */
145  /* creation of BlinkLed1Thread */
146  BlinkLed1ThreadHandle = osThreadNew(BlinkLed1, NULL, &BlinkLed1Thread_attributes);
147
148  /* creation of BlinkLed2Thread */
149  BlinkLed2ThreadHandle = osThreadNew(BlinkLed2, NULL, &BlinkLed2Thread_attributes);

159  /* Start scheduler */
160  osKernelStart();
161
162  /* We should never get here as control is now taken by the scheduler */
163  /* Infinite loop */
164  /* USER CODE BEGIN WHILE */
165  while (1)
166  {
```



Blinky blinky

```
407 /* USER CODE END Header_BlinkLed1 */
408 void BlinkLed1(void *argument)
409 {
410     /* USER CODE BEGIN 5 */
411     /* Infinite loop */
412     // OPM: om een oneindig lang lopende lus te maken, kan je uiteraard ook while(1) gebruiken.
413     //      Dikwijls zijn er 'code conventions' binnen een bedrijf die je aangeven wat je best gebruikt.
414     //      Het uiteindelijke resultaat van beide lussen is gelijk.
415     for(;;)
416     {
417         // LED 1 toggle'n.
418         HAL_GPIO_TogglePin(LED1_GPIO_Port, LED1_Pin);
419
420         // OPM: osDelay() plaatst de thread in 'blocked' state voor de bepaalde tijd. Dat zorgt ervoor dat
421         //      er geen CPU-tijd naar dat proces gaat.
422         //      Mocht je hier HAL_Delay(500) gebruiken, dan zal de thread toch de aandacht van de CPU nodig
423         //      hebben tijdens het wachten... Als je dus efficiënt wil werken in RTOS, gebruik je meestal
424         //      osDelay()...
425         osDelay(500);
426     }
427     /* USER CODE END 5 */
428 }
```



Blinky blinky

```
436 /* USER CODE END Header_BlinkLed2 */
437 void BlinkLed2(void *argument)
438 {
439     /* USER CODE BEGIN BlinkLed2 */
440     /* Infinite loop */
441     for(;;)
442     {
443         // LED 2 toggle'n.
444         HAL_GPIO_TogglePin(LED2_GPIO_Port, LED2_Pin);
445
446         // OPM: osDelay() plaatst de thread in 'blocked' state voor de bepaalde tijd. Dat zorgt ervoor dat
447         //       er geen CPU-tijd naar dat proces gaat.
448         osDelay(123);
449     }
450     /* USER CODE END BlinkLed2 */
451 }
```



Blinky blinky

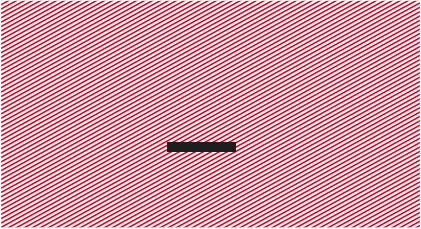
Tot slot: hieronder de code die de **globale timer voor de HAL-functies** verhoogt. Vroeger, zonder RTOS, werd dat gedaan in de SysTick handler. Nu is die SysTick bezet door het RTOS en moeten we bijvoorbeeld Timer6 gebruiken daarvoor.

```
461 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
462 {
463     /* USER CODE BEGIN Callback 0 */
464
465     /* USER CODE END Callback 0 */
466     if (htim->Instance == TIM6) {
467         HAL_IncTick();
468     }
469     /* USER CODE BEGIN Callback 1 */
470
471     /* USER CODE END Callback 1 */
472 }
```



Blinky blinky

Compileer het voorbeeld en **controleer de werking** van de twee LED's op het Nucleo Extension Shield...



Blinky blinky met UART2 TX



Blinky blinky met UART2 TX

Doel?

Een codevoorbeeld maken met CMSIS RTOS V2 waarbij de **UART** gebruikt wordt.

Hoe?

Gebruik STM32CubeMX om een CMSIS RTOS V2 project aan te maken met twee threads.

De ene thread laat een LED knipperen. De andere **verzendt** iedere seconde info over **USART2**...

Blinky blinky met UART2 TX

STM32CubeMX CubeMX - Template - Nucleo-L476RG.ioc: STM32L476RGTx NUCLEO-L476RG

File Window Help myST

Home STM32L476RGTx - NUCLEO-L476RG CubeMX - Template - Nucleo-L476RG.ioc - Pinout & Configuration GENERATE CODE

Pinout & Configuration Clock Configuration Project Manager Tools

Software Packs Pinout

Categories A-Z

Middleware and Software Pa...

AIROC-Wi-Fi-Bluetooth-ST...
FATFS
FP-ATR-SIGFOX1
FP-SNS-FLIGHT1
FP-SNS-MOTENV1
FP-SNS-SMARTAG2
FREERTOS
I-CUBE-Cesium
I-CUBE-ITTIADB
I-CUBE-embOS
I-CUBE-wolfSSL
I-Cube-SoM-uGOAL
TOUCHSENSING
USB_DEVICE
USB_HOST
X-CUBE-AI
X-CUBE-ALGOBUILD
X-CUBE-ALS
X-CUBE-AZRTOS-L4
X-CUBE-BLE1
X-CUBE-BLE2
X-CUBE-BLEMGR
X-CUBE-DISPLAY
X-CUBE-EEPRMA1
X-CUBE-GNSS1
X-CUBE-ISPU
X-CUBE-MEMS1
X-CUBE-NFC4
X-CUBE-NFC6

FREERTOS Mode and Configuration

Interface CMSIS_V2

Configuration

Reset Configuration

Tasks and Queues Timers and Semaphores Mutexes Events FreeRTOS Heap Usage

Config parameters Include parameters Advanced settings User Constants

Tasks

Task Name	Priority	Stack Siz...	Entry Function	Code Generati...	Parameter	Allocation	Buffer Name	Control Block ...
BlinkLed1Thread	osPriorityNormal	128	BlinkLed1	Default	NULL	Dynamic	NULL	NULL
Usart2TxThread	osPriorityNormal	128	Usart2Tx	Default	NULL	Dynamic	NULL	NULL

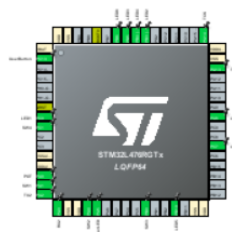
Add Delete

Queues

Queue Name	Queue Size	Item Size	Allocation	Buffer Name	Control Block Name
------------	------------	-----------	------------	-------------	--------------------

Add Delete

Pinout view System view



Blinky blinky met UART2 TX

The screenshot shows the STM32CubeMX software interface. The 'Pinout & Configuration' tab is active, displaying the 'USART2 Mode and Configuration' settings. In the left-hand 'Categories' list, 'USART2' is highlighted with a red box. The main configuration area shows the following settings:

- Mode:** Asynchronous
- Hardware Flow Control (RS232):** Disable
- Hardware Flow Control (RS485):** ☒ Enabled

The 'Configuration' section includes a 'Reset Configuration' button and tabs for 'Parameter Settings', 'User Constants', 'NVIC Settings', 'DMA Settings', and 'GPIO Settings'. The 'Parameter Settings' tab is selected, showing the following parameters:

Parameter	Value
Baud Rate	9600 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1
Data Direction	Transmit Only
Over Sampling	16 Samples
Single Sample	Disable
Auto Baudrate	Disable
TX Pin Active Level Inversion	Disable
RX Pin Active Level Inversion	Disable
Data Inversion	Disable
TX and RX Pins Swapping	Disable
Overrun	Disable
DMA on RX Error	Enable

On the right side of the window, the 'Pinout view' shows a diagram of the STM32L476RG microcontroller with the USART2 pins highlighted.

Blinky blinky met UART2 TX

```
53 /* Definitions for BlinkLed1Thread */
54 osThreadId_t BlinkLed1ThreadHandle;
55 const osThreadAttr_t BlinkLed1Thread_attributes = {
56     .name = "BlinkLed1Thread",
57     .stack_size = 128 * 4,
58     .priority = (osPriority_t) osPriorityNormal,
59 };
60 /* Definitions for Usart2TxThread */
61 osThreadId_t Usart2TxThreadHandle;
62 const osThreadAttr_t Usart2TxThread_attributes = {
63     .name = "Usart2TxThread",
64     .stack_size = 128 * 4,
65     .priority = (osPriority_t) osPriorityNormal,
66 };

143 /* creation of BlinkLed1Thread */
144 BlinkLed1ThreadHandle = osThreadNew(BlinkLed1, NULL, &BlinkLed1Thread_attributes);
145
146 /* creation of Usart2TxThread */
147 Usart2TxThreadHandle = osThreadNew(Usart2Tx, NULL, &Usart2TxThread_attributes);
```

Blinky blinky met UART2 TX

```
406 void BlinkLed1(void *argument)
407 {
408     /* USER CODE BEGIN 5 */
409     /* Infinite loop */
410     for(;;)
411     {
412         // User LED toggle'n.
413         HAL_GPIO_TogglePin(UserLED_GPIO_Port, UserLED_Pin);
414
415         // Even wachten (in Blocked status).
416         osDelay(500);
417     }
418     /* USER CODE END 5 */
419 }
```

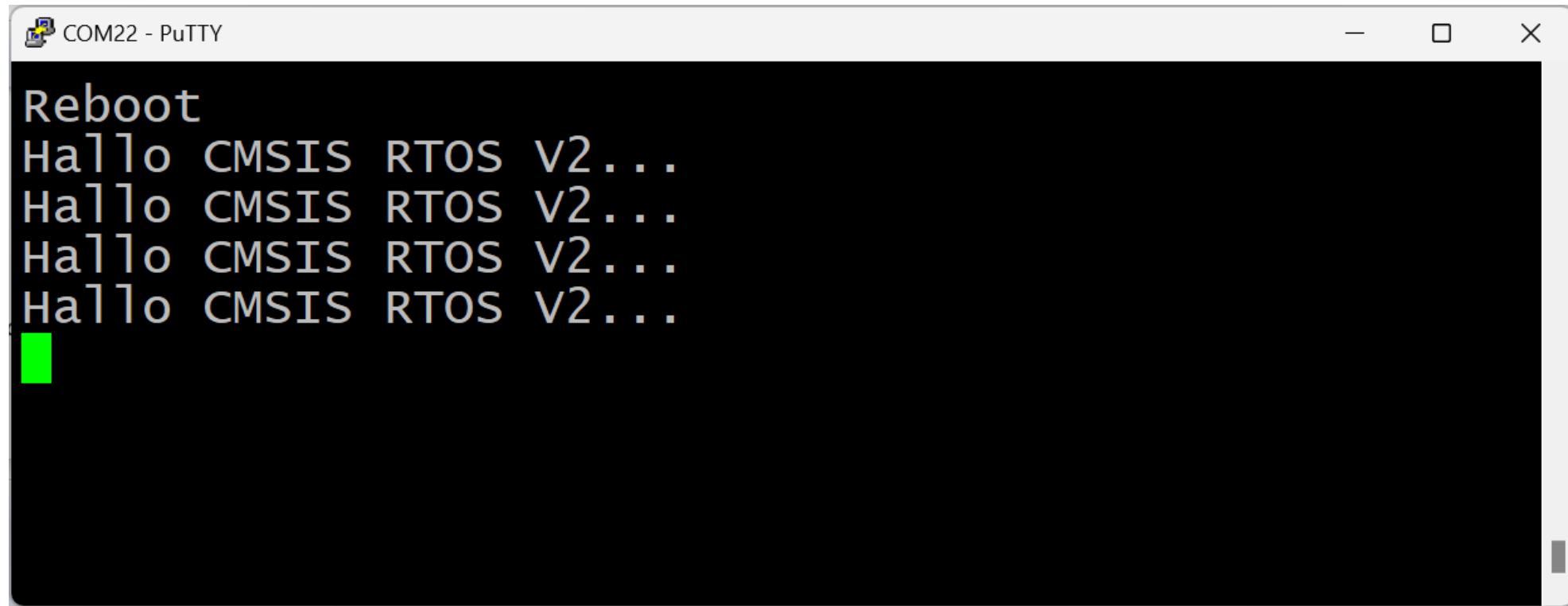


Blinky blinky met UART2 TX

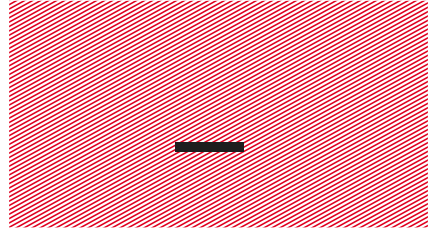
```
428 void Usart2Tx(void *argument)
429 {
430     /* USER CODE BEGIN Usart2Tx */
431     /* Infinite loop */
432     for(;;)
433     {
434         // Teken van leven geven via UART2.
435         HAL_UART_Transmit(&huart2, (uint8_t*)MESSAGE, strlen(MESSAGE), HAL_MAX_DELAY);
436
437         // Even wachten.
438         osDelay(1000);
439     }
440     /* USER CODE END Usart2Tx */
441 }
```

—

Blinky blinky met UART2 TX



```
COM22 - PuTTY
Reboot
Hallo CMSIS RTOS V2...
Hallo CMSIS RTOS V2...
Hallo CMSIS RTOS V2...
Hallo CMSIS RTOS V2...
█
```



Blinky blinky met UART2 TX en RX



Blinky blinky met UART2 TX en RX

Doel?

Een codevoorbeeld maken met CMSIS RTOS V2 waarbij de **UART gebruikt wordt**.

Hoe?

Gebruik STM32CubeMX om een CMSIS RTOS V2 project aan te maken met **drie threads**.

De eerste thread laat een **LED knipperen**. De tweede thread **verzendt** iedere seconde info over USART2... De laatste thread wacht continu op **info** die via USART2 kan **binnenkomen**....

Blinky blinky met UART2 TX en RX

STM32CubeMX CubeMX - Template - Nucleo-L476RG.ioc*: STM32L476RGTx NUCLEO-L476RG

File Window Help myST

Home > STM32L476RGTx - NUCLEO-L476RG > CubeMX - Template - Nucleo-L476RG.ioc - Pinout & Configuration > GENERATE CODE

Pinout & Configuration

Software Packs Pinout

FREERTOS Mode and Configuration

Mode

Interface CMSIS_V2

Configuration

Reset Configuration

✓ User Constants	✓ Tasks and Queues	✓ Timers and Semaphores	✓ Mutexes	✓ Events	✓ FreeRTOS Heap Usage
✓ Config parameters	✓ Include parameters	✓ Advanced settings			

Tasks

Task Name	Priority	Stack Size (Words)	Entry Function	Code Generation Opt...	Parameter	Allocation	Buffer Name	Control Block Name
BlinkLed1Thread	osPriorityNormal	128	BlinkLed1	Default	NULL	Dynamic	NULL	NULL
Usart2TxThread	osPriorityNormal	128	Usart2Tx	Default	NULL	Dynamic	NULL	NULL
Usart2RxThread	osPriorityNormal	128	Usart2Rx	Default	NULL	Dynamic	NULL	NULL

Add Delete

Queues

Queue Name	Queue Size	Item Size	Allocation	Buffer Name	Control Block Name
------------	------------	-----------	------------	-------------	--------------------

Add Delete

Categories A-Z

Middleware and Software Pa...

- AIROC-Wi-Fi-Bluetooth-ST...
- FATFS
- FP-ATR-SIGFOX1
- FP-SNS-FLIGHT1
- FP-SNS-MOTENV1
- FP-SNS-SMARTAG2
- ✓ FREERTOS
- I-CUBE-Cesium
- I-CUBE-ITTIADB
- I-CUBE-embOS
- I-CUBE-wolfSSL
- I-Cube-SoM-uGOAL
- TOUCHSENSING
- USB_DEVICE
- USB_HOST
- X-CUBE-AI
- X-CUBE-ALGOBUILD
- X-CUBE-ALS
- X-CUBE-AZRTOS-L4
- X-CUBE-BLE1
- X-CUBE-BLE2
- X-CUBE-BLEMGR
- X-CUBE-DISPLAY
- X-CUBE-EEPRMA1
- X-CUBE-GNSS1
- X-CUBE-ISPUI
- X-CUBE-MEMS1
- X-CUBE-NFC4
- X-CUBE-NFC6

Blinky blinky met UART2 TX en RX

STM32CubeMX CubeMX - Template - Nucleo-L476RG.ioc*: STM32L476RGtx NUCLEO-L476RG

File Window Help myST

Home STM32L476RGtx - NUCLEO-L476RG CubeMX - Template - Nucleo-L476RG.ioc - Pinout & Configuration GENERATE CODE

Pinout & Configuration Clock Configuration Project Manager Tools

Software Packs Pinout

USART2 Mode and Configuration

Mode

Mode Asynchronous

Hardware Flow Control (RS232) Disable

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

Basic Parameters

Baud Rate 9600 Bits/s

Word Length 8 Bits (including Parity)

Parity None

Stop Bits 1

Advanced Parameters

Data Direction Receive and Transmit

Over Sampling 16 Samples

Single Sample

Advanced Features

Auto Baudrate Disable

TX Pin Active Level Inversion Disable

RX Pin Active Level Inversion Disable

Data Inversion Disable

TX and RX Pins Swapping Disable

Overrun Disable

DMA on RX Error Enable

MSB First Disable

Categories A-Z

Timers

Connectivity

CAN1

I2C1

I2C2

I2C3

IRTIM

LPUART1

QUADSPI

SDMMC1

SPI1

SPI2

SPI3

SWPMI1

UART4

UART5

USART1

USART2

USART3

USB_OTG_FS

Multimedia

Security

Computing

Blinky blinky met UART2 TX en RX

```

53  /* Definitions for BlinkLed1Thread */
54  osThreadId_t BlinkLed1ThreadHandle;
55  const osThreadAttr_t BlinkLed1Thread_attributes = {
56      .name = "BlinkLed1Thread",
57      .stack_size = 128 * 4,
58      .priority = (osPriority_t) osPriorityNormal,
59  };
60  /* Definitions for Usart2TxThread */
61  osThreadId_t Usart2TxThreadHandle;
62  const osThreadAttr_t Usart2TxThread_attributes = {
63      .name = "Usart2TxThread",
64      .stack_size = 128 * 4,
65      .priority = (osPriority_t) osPriorityNormal,
66  };
67  /* Definitions for Usart2RxThread */
68  osThreadId_t Usart2RxThreadHandle;
69  const osThreadAttr_t Usart2RxThread_attributes = {
70      .name = "Usart2RxThread",
71      .stack_size = 128 * 4,
72      .priority = (osPriority_t) osPriorityNormal,
73  };

151  /* creation of BlinkLed1Thread */
152  BlinkLed1ThreadHandle = osThreadNew(BlinkLed1, NULL, &BlinkLed1Thread_attributes);
153
154  /* creation of Usart2TxThread */
155  Usart2TxThreadHandle = osThreadNew(Usart2Tx, NULL, &Usart2TxThread_attributes);
156
157  /* creation of Usart2RxThread */
158  Usart2RxThreadHandle = osThreadNew(Usart2Rx, NULL, &Usart2RxThread_attributes);

```

Blinky blinky met UART2 TX en RX

```
417 void BlinkLed1(void *argument)
418 {
419     /* USER CODE BEGIN 5 */
420     /* Infinite loop */
421     for(;;)
422     {
423         // User LED toggle'n.
424         HAL_GPIO_TogglePin(UserLED_GPIO_Port, UserLED_Pin);
425
426         // Even wachten (in Blocked status).
427         osDelay(500);
428     }
```

```
439 void Usart2Tx(void *argument)
440 {
441     /* USER CODE BEGIN Usart2Tx */
442     /* Infinite loop */
443     for(;;)
444     {
445         // Teken van leven geven via UART2.
446         HAL_UART_Transmit(&huart2, (uint8_t*)MESSAGE, strlen(MESSAGE), HAL_MAX_DELAY);
447
448         // Even wachten.
449         osDelay(1000);
450     }
451     /* USER CODE END Usart2Tx */
452 }
```

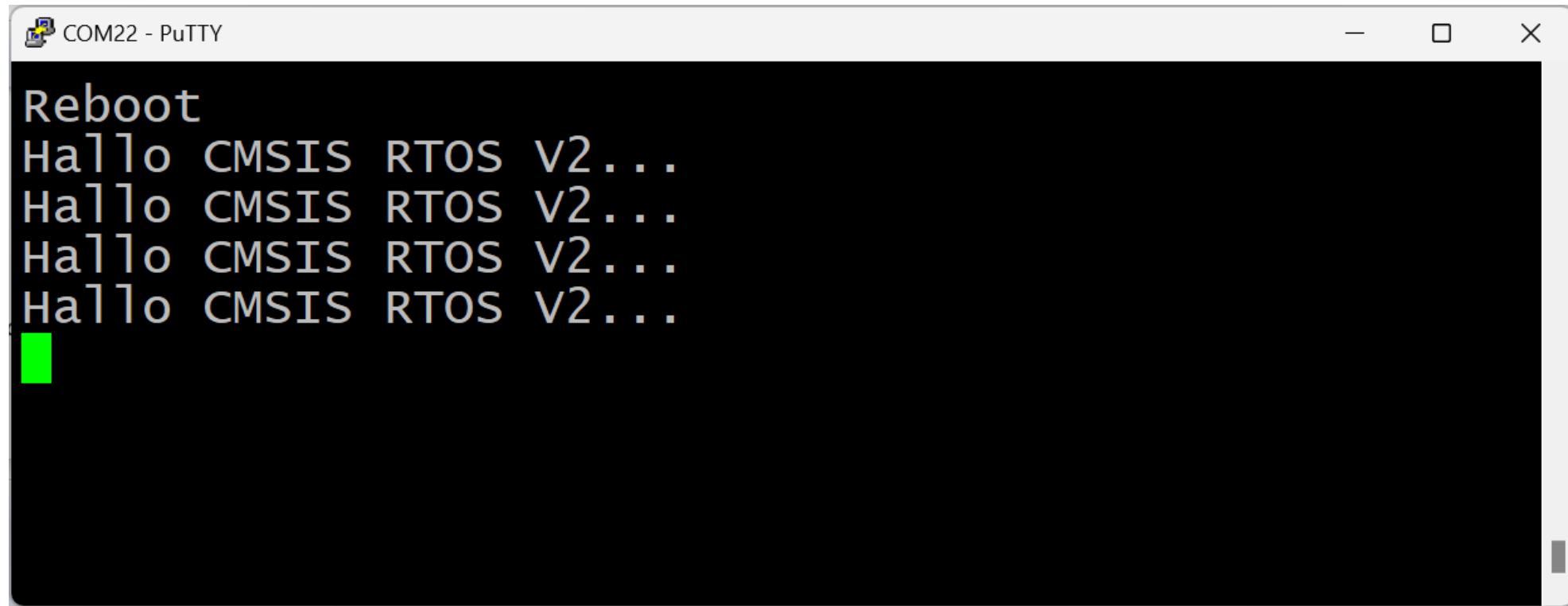


Blinky blinky met UART2 TX en RX

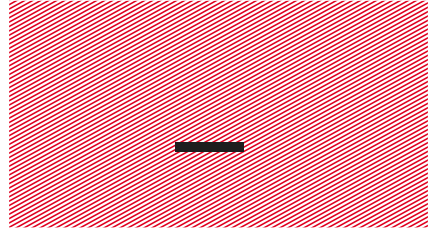
```
461 void Usart2Rx(void *argument)
462 {
463     /* USER CODE BEGIN Usart2Rx */
464     uint8_t receivedData = 0;
465
466     /* Infinite loop */
467     for(;;)
468     {
469         // Probeer één byte te ontvangen via USART2. Doe dat via 'polling' en wacht oneindig lang...
470         // Zonder het concept threads (tasks) in een RTOS, zou deze code blokkeren tot er een byte
471         // ontvangen wordt. Nu zal deze thread de voorziene tijd krijgen van de scheduler, daarna wordt
472         // overgestapt naar een andere thread (indien nodig). Deze thread krijgt daarna opnieuw de aandacht
473         // van de CPU om te kijken of er één byte is aangekomen. Deze afwisseling van threads herhaalt zich
474         // continu.
475         HAL_UART_Receive(&huart2, (uint8_t*)&receivedData, 1, HAL_MAX_DELAY);
476
477         // Als er een byte ontvangen is, toon hem op de LED's.
478         ByteToLeds(receivedData);
479     }
```

—

Blinky blinky met UART2 TX en RX



```
COM22 - PuTTY
Reboot
Hallo CMSIS RTOS V2...
Hallo CMSIS RTOS V2...
Hallo CMSIS RTOS V2...
Hallo CMSIS RTOS V2...
█
```



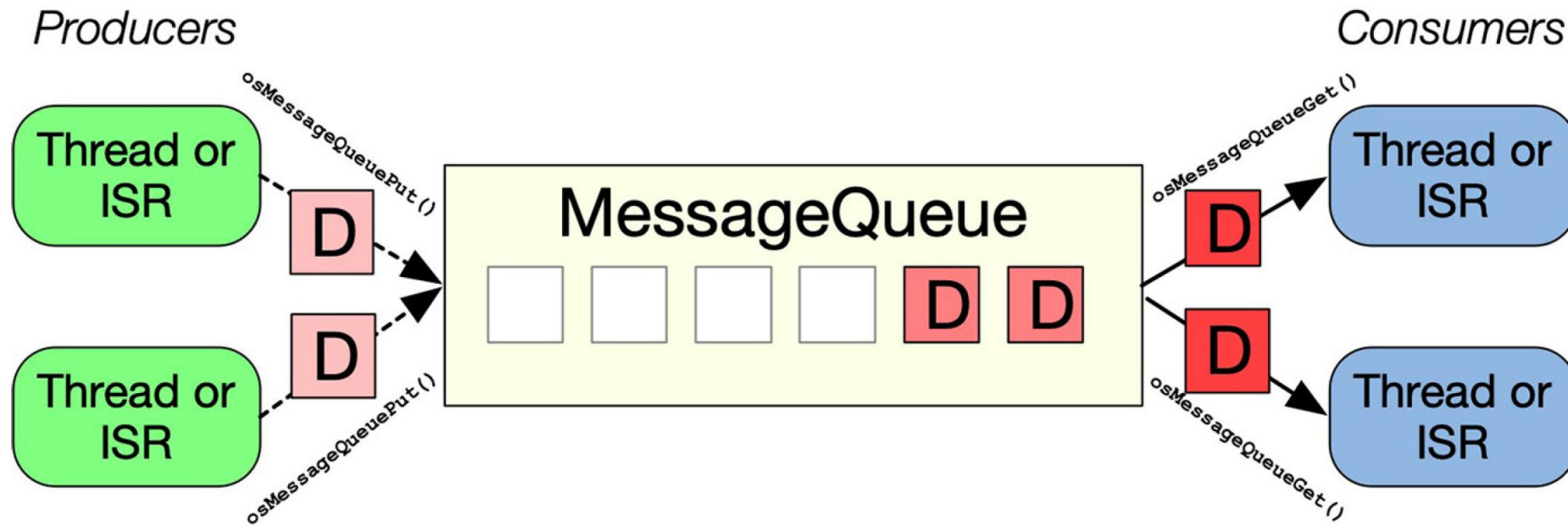
Message queue



Message queue

- In een multi-threaded applicatie, zal je vroeg of laat **een vorm van synchronisatie** nodig hebben tussen die threads.
- Dat kan bijvoorbeeld de **toegang** zijn tot **gedeelde bronnen**, bijvoorbeeld in het geheugen of een UART.
- Een eerste 'synchronization primitive' is de **message queue**.
- Vrij vertaald: een **berichtenwachtrij**.
- Zo'n message queue is een First In First Out (**FIFO**) structuur.
- Je kan elementen toevoegen aan het begin van de rij en elementen verwijderen aan het eind van de rij.

Message queue



Je kan dus bijvoorbeeld twee threads hebben, die één gedeelde message queue gebruiken **om data uit te wisselen**... Het RTOS beveiligt ook tegen schrijven naar een volle buffer of lezen uit een lege buffer. Op dat moment worden de threads in '**blocked mode**' gezet.

Message queue

STM32CubeMX ondersteunt het **maken van queues**. Je kan dus daar de wachtrijen voorbereiden en dan in μ Vision gebruiken...

The screenshot shows the STM32CubeMX configuration interface. The 'Tasks and Queues' tab is selected. Below the 'Tasks' table, the 'Queues' section is highlighted with a red box. It contains a table with the following data:

Queue Name	Queue Size	Item Size	Allocation	Buffer Name	Control Block Name
messageQueue	3	uint16_t	Dynamic	NULL	NULL

Message queue

Het aanmaken van een nieuwe queue kan in C-taal met de functie *osMessageQueueNew()*.

```
osMessageQueueId_t osMessageQueueNew ( uint32_t      msg_count,  
                                       uint32_t      msg_size,  
                                       const osMessageQueueAttr_t * attr  
                                       )
```

Parameters

- [in] **msg_count** maximum number of messages in queue.
- [in] **msg_size** maximum message size in bytes.
- [in] **attr** message queue attributes; NULL: default values.

Returns

message queue ID for reference by other functions or NULL in case of error.

The function **osMessageQueueNew** creates and initializes a message queue object. The function returns a message queue object identifier or **NULL** in case of an error.

The function can be called after kernel initialization with **osKernelInitialize**. It is possible to create message queue objects before the RTOS kernel is started with **osKernelStart**.

The total amount of memory required for the message queue data is at least `msg_count * msg_size`. The `msg_size` is rounded up to a double even number to ensure 32-bit alignment of the memory blocks.

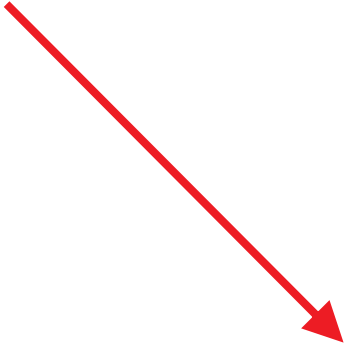
The memory blocks allocated from the message queue have a fixed size defined with the parameter `msg_size`.

Bron: https://arm-software.github.io/CMSIS_6/latest/RTOS2/group_CMSIS_RTOS_Message.html#ga24e895a00f9d484db33aaf784c57bfed

Het aanmaken van een nieuwe queue kan met de functie *osMessageQueueNew()*.

```
67 /* Definitions for messageQueue */
68 osMessageQueueId_t messageQueueHandle;
69 const osMessageQueueAttr_t messageQueue_attributes = {
70     .name = "messageQueue"
71 };
```

```
143 /* Create the queue(s) */
144 /* creation of messageQueue */
145 messageQueueHandle = osMessageQueueNew (3, sizeof(uint16_t), &messageQueue_attributes);
```



Message queue

```
53 /* Definitions for RunningLightsTh */
54 osThreadId_t RunningLightsThHandle;
55 const osThreadAttr_t RunningLightsTh_attributes = {
56     .name = "RunningLightsTh",
57     .stack_size = 128 * 4,
58     .priority = (osPriority_t) osPriorityNormal,
59 };
60 /* Definitions for Usart2Thread */
61 osThreadId_t Usart2ThreadHandle;
62 const osThreadAttr_t Usart2Thread_attributes = {
63     .name = "Usart2Thread",
64     .stack_size = 128 * 4,
65     .priority = (osPriority_t) osPriorityNormal,
66 };
```

```
151 /* Create the thread(s) */
152 /* creation of RunningLightsTh */
153 RunningLightsThHandle = osThreadNew(RunningLights, NULL, &RunningLightsTh_attributes);
154
155 /* creation of Usart2Thread */
156 Usart2ThreadHandle = osThreadNew(Usart2, NULL, &Usart2Thread_attributes);
```

Message queue

enqueueing

```
457 void Usart2(void *argument)
458 {
459     /* USER CODE BEGIN Usart2 */
460     uint16_t requestedDelay = 0;
461
462     /* Infinite loop */
463     for(;;)
464     {
465         // Vragen aan de gebruiker wat de gewenste wachttijd (delay) moet zijn voor het looplicht.
466         printf("Wat is de gewenste wachttijd (groter dan 0) voor het looplicht (in milliseconden)?\r\n");
467         scanf("%hu", &requestedDelay); // %hu vraagt een unsigned short int op
468                                         // (https://www.tutorialspoint.com/c_standard_library/c_function_scanf.htm).
469
470         // Beveiligen tegen '0' wachttijd.
471         if(requestedDelay == 0)
472             requestedDelay = 1;
473
474         // Melding maken van het opgeslagen getal.
475         // %hu toont een unsigned short int (https://www.tutorialspoint.com/c_standard_library/c_function_printf.htm).
476         printf("De ingelezen gewenste wachttijd is: %hu ms.\r\n", requestedDelay);
477
478         // Nieuwe waarde opslaan in een Message Queue, zodat die gelezen kan worden door een andere thread.
479         // Ook hier is de prioriteit niet geïmplementeerd en hoeft je er dus niets mee te doen.
480         osMessageQueuePut(messageQueueHandle, &requestedDelay, 0, osWaitForever);
481
482         // Melden dat de queue is aangevuld. Probeer maar eens vrij snel nieuwe waarden in te voeren. Dan
483         // is te zien dat onderstaande melding op zich laat wachten als er geen plaats meer is in de queue.
484         // Info over de gebruikte functies,
485         // zie: https://arm-software.github.io/CMSIS_5/RTOS2/html/group__CMSIS__RTOS__Message.html.
486         printf("De queue werd aangevuld (%d/%d).\r\n\r\n", osMessageQueueGetCount(messageQueueHandle),
487             osMessageQueueGetCapacity(messageQueueHandle));
488     }
489     /* USER CODE END Usart2 */
490 }
```

Message queue

dequeueing

```
413 /* USER CODE END Header_RunningLights */
414 void RunningLights(void *argument)
415 {
416     /* USER CODE BEGIN 5 */
417     uint8_t count = 0, leds = 0x01;
418     uint16_t delay = 100;
419     uint16_t message = 0;
420     osStatus_t status = osOK;
421
422     /* Infinite loop */
423     for(;;)
424     {
425         // OPM: message priority is nog niet geïmplementeerd in FreeRTOS/CMSIS RTOS V2. Dat argument mag op 0 gezet worden.
426         status = osMessageQueueGet(messageQueueHandle, &message, 0, 5);
427
428         // Is er een nieuw bericht ontvangen? Ga verder met die nieuwe waarde.
429         if(status == osOK)
430             delay = message;
431
432         // Looplicht maken.
433         for(count = 0; count < 8; count++)
434         {
435             ByteToLeds(leds);
436
437             leds <<= 1;
438             if(leds == 0x00)
439                 leds = 0x01;
440
441             // Even wachten (in Blocked status).
442             osDelay(delay);
443         }
444     }
445     /* USER CODE END 5 */
446 }
```


Message queue

```
COM22 - PuTTY
Reboot

Wat is de gewenste wachttijd (groter dan 0) voor het looplicht (in milliseconden)?
De ingelezen gewenste wachttijd is: 1234 ms.
De queue werd aangevuld (1/3).

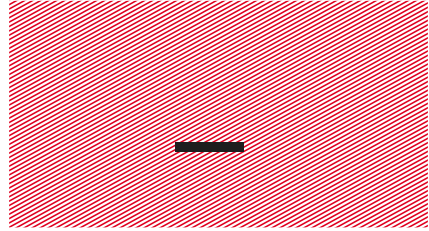
Wat is de gewenste wachttijd (groter dan 0) voor het looplicht (in milliseconden)?
De ingelezen gewenste wachttijd is: 2345 ms.
De queue werd aangevuld (1/3).

Wat is de gewenste wachttijd (groter dan 0) voor het looplicht (in milliseconden)?
De ingelezen gewenste wachttijd is: 3456 ms.
De queue werd aangevuld (2/3).

Wat is de gewenste wachttijd (groter dan 0) voor het looplicht (in milliseconden)?
De ingelezen gewenste wachttijd is: 4321 ms.
De queue werd aangevuld (2/3).

Wat is de gewenste wachttijd (groter dan 0) voor het looplicht (in milliseconden)?
De ingelezen gewenste wachttijd is: 5432 ms.
De queue werd aangevuld (3/3).

Wat is de gewenste wachttijd (groter dan 0) voor het looplicht (in milliseconden)?
De ingelezen gewenste wachttijd is: 654 ms.
De queue werd aangevuld (3/3).
```



Theorieopdracht





Theorieopdracht

Maak gebruik van de demo code i.v.m. het aanmaken van een RTOS-project voor UART2 om enkele vragen te beantwoorden.

De code heeft als naam: 'Nucleo-L476RG - CMSIS RTOS V2 - Blinky UART2 TX RX'.

Zie:

"Theorieopdracht 7 – CMSIS RTOS V2 – Blinky UART2 TX RX".